

The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying a project structure with folders for Week_1, Week_2, Week_3, and an output folder. The file explorer is open to '1 MergSort.cpp'. The main editor window shows the implementation of the merge function in C++:

```
Week_3 > 1 MergSort.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  void merge(int arr[], int left, int mid, int right)
5  {
6      int n1 = mid - left + 1;
7      int n2 = right - mid;
8      int L[n1], R[n2];
9
10     for (int i = 0; i < n1; i++)
11         L[i] = arr[left + i];
12     for (int j = 0; j < n2; j++)
13         R[j] = arr[mid + 1 + j];
14
15     int i = 0, j = 0, k = left;
16     while (i < n1 && j < n2)
17     {
18         if (L[i] <= R[j])
19         {
20             arr[k] = L[i];
21             i++;
22         }
23         else
24         {
25             arr[k] = R[j];
26             j++;
27         }
28         k++;
29     }
30     while(i < n1)
31     {
32         arr[k] = L[i];
33         i++;
```

The status bar at the bottom indicates the current position is Ln 70, Col 2, with 4 spaces, UTF-8 encoding, CRLF line endings, and C++ language.

The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying a project structure with folders for Week_1, Week_2, Week_3, and an output folder. The file explorer is open to '1 MergSort.cpp'. The main editor window shows the implementation of the mergeSort function and the main function in C++:

```
Week_3 > 1 MergSort.cpp > mergeSort(int [], int, int)
38     arr[k] = R[j];
39     j++;
40     k++;
41 }
42 }
43 void mergeSort(int arr[], int left, int right)
44 {
45     if (left < right)
46     {
47         int mid = (left + right) / 2;
48         mergeSort(arr, left, mid);
49         mergeSort(arr, mid + 1, right);
50         merge(arr, left, mid, right);
51     }
52 }
53 int main()
54 {
55     int N;
56     cin >> N;
57     int arr[N];
58
59     for (int i = 0; i < N; i++)
60         cin >> arr[i];
61     mergeSort(arr, 0, N - 1);
62 }
```

The status bar at the bottom indicates the current position is Ln 52, Col 2, with 4 spaces, UTF-8 encoding, CRLF line endings, and C++ language.

Below the editor, the terminal window shows the output of the program:

```
C/C++ Compile Run - output
454
656
6
5
5
5 5 6 454 656
PS (thisisdurgeshkumar.com): C:\Users\pc\OneDrive\Desktop\Module\Week_3\output> 65
```

This screenshot shows the Visual Studio Code editor with the file `2 MergeSort.cpp` open. The code implements the `merge` function, which takes an array `arr` and indices `left`, `mid`, and `right` as parameters. It creates two temporary arrays `L` and `R` to store the left and right halves of the array. The function then merges these two halves back into the original array `arr` by comparing elements from `L` and `R` and placing them in `arr` in sorted order.

```
1 #include <iostream>
2 using namespace std;
3 void merge(int arr[], int left, int mid, int right)
4 {
5     int n1 = mid - left + 1;
6     int n2 = right - mid;
7
8     int L[n1], R[n2];
9     for(int i = 0; i < n1; i++)
10         L[i] = arr[left + i];
11
12     for(int j = 0; j < n2; j++)
13         R[j] = arr[mid + 1 + j];
14
15     int i = 0, j = 0, k = left;
16     while(i < n1 && j < n2)
17     {
18         if(L[i] <= R[j])
19             arr[k++] = L[i++];
20         else
21             arr[k++] = R[j++];
22     }
23     while(i < n1)
24         arr[k++] = L[i++];
25     while(j < n2)
26         arr[k++] = R[j++];
27 }
28
29 // Merge Sort Function
30 void mergeSort(int arr[], int left, int right)
31 {
32     if(left < right)
33     {
```

This screenshot shows the Visual Studio Code editor with the file `2 MergeSort.cpp` open. The code implements the `main` function, which reads an integer `N` from the user, creates an array `arr` of size `N`, and fills it with random values. It then calls `mergeSort(arr, 0, N - 1)` to sort the array. Finally, it prints the sorted array and the median value.

```
36 mergeSort(arr, left, mid);
37 mergeSort(arr, mid + 1, right);
38
39 merge(arr, left, mid, right);
40 }
41
42 int main()
43 {
44     int N;
45     cin >> N;
46
47     int arr[N];
48
49     for(int i = 0; i < N; i++)
50         cin >> arr[i];
51     mergeSort(arr, 0, N - 1);
52
53     cout << "Sorted Processing Times:\n";
54     for(int i = 0; i < N; i++)
55         cout << arr[i] << " ";
56
57     cout << endl;
58     int median = arr[N / 2];
59     cout << "Median Processing Time: " << median << endl;
60 }
```

The output of the program is shown in the terminal window at the bottom of the editor:

```
56
76
Sorted Processing Times:
6 54 56 76 4576
Median Processing Time: 56
Orders Greater than Median: 2
Difference: 4570
```

```
1_HeapSort.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  void heapify(int arr[], int n, int i)
5  {
6      int largest = i;
7      int left = 2 * i + 1;
8      int right = 2 * i + 2;
9      if (left < n && arr[left] > arr[largest])
10         largest = left;
11
12     if (right < n && arr[right] > arr[largest])
13         largest = right;
14     if (largest != i)
15     {
16         swap(arr[i], arr[largest]);
17         heapify(arr, n, largest);
18     }
19 }
20
21 // Heap Sort
22 void heapSort(int arr[], int n)
23 {
24     for (int i = n / 2 - 1; i >= 0; i--)
25         heapify(arr, n, i);
26
27     for (int i = n - 1; i > 0; i--)
28     {
29         swap(arr[0], arr[i]);
30         heapify(arr, i, 0);
31     }
32 }
33
```

```
1_HeapSort.cpp > main()
21 // Heap Sort
22 void heapSort(int arr[], int n)
23 {
24     for (int i = n / 2 - 1; i >= 0; i--)
25         heapify(arr, n, i);
26
27     for (int i = n - 1; i > 0; i--)
28     {
29         swap(arr[0], arr[i]);
30         heapify(arr, i, 0);
31     }
32 }
33
34 int main()
35 {
36     int N;
37     cin >> N;
38     int arr[N];
39     for (int i = 0; i < N; i++)
40         cin >> arr[i];
41
42     heapSort(arr, N);
43     for (int i = 0; i < N; i++)
44         cout << arr[i] << " ";
45 }
```

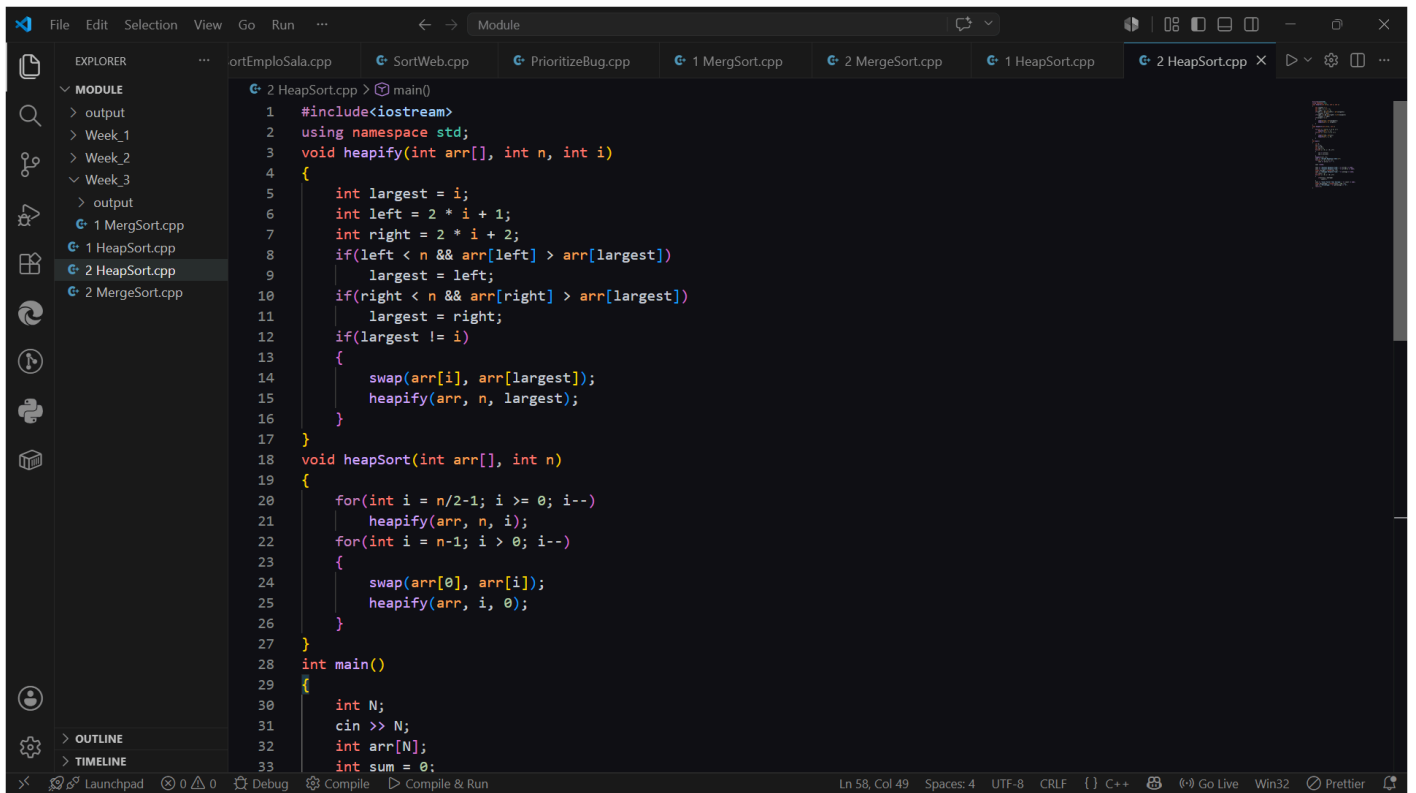
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

4

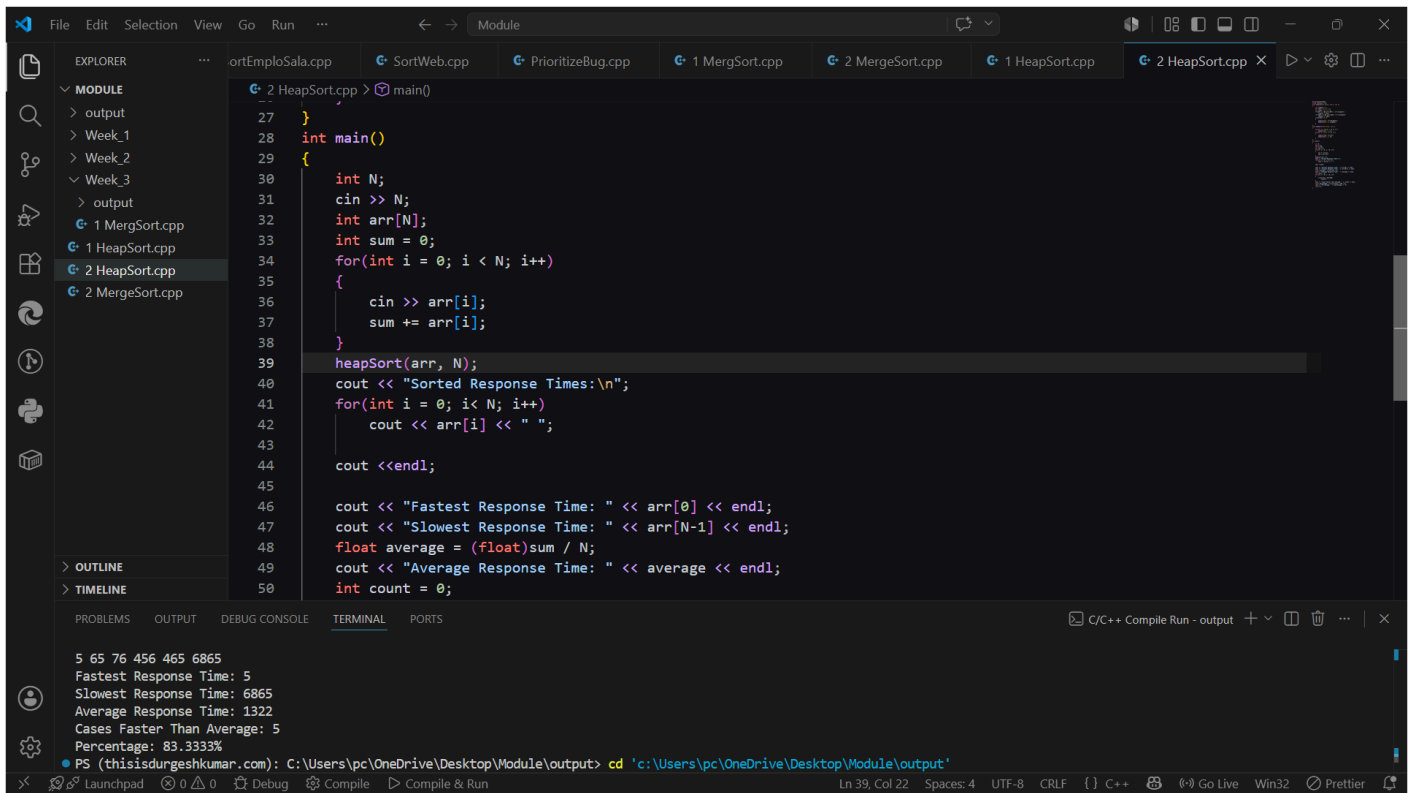
566
565
656

4 46 565 566 656

PS (thisisdurgeshkumar.com): C:\Users\pc\OneDrive\Desktop\Module\output>



```
1 #include<iostream>
2 using namespace std;
3 void heapify(int arr[], int n, int i)
4 {
5     int largest = i;
6     int left = 2 * i + 1;
7     int right = 2 * i + 2;
8     if(left < n && arr[left] > arr[largest])
9         largest = left;
10    if(right < n && arr[right] > arr[largest])
11        largest = right;
12    if(largest != i)
13    {
14        swap(arr[i], arr[largest]);
15        heapify(arr, n, largest);
16    }
17 }
18 void heapSort(int arr[], int n)
19 {
20     for(int i = n/2-1; i >= 0; i--)
21         heapify(arr, n, i);
22     for(int i = n-1; i > 0; i--)
23     {
24         swap(arr[0], arr[i]);
25         heapify(arr, i, 0);
26     }
27 }
28 int main()
29 {
30     int N;
31     cin >> N;
32     int arr[N];
33     int sum = 0;
```



```
27 }
28 int main()
29 {
30     int N;
31     cin >> N;
32     int arr[N];
33     int sum = 0;
34     for(int i = 0; i < N; i++)
35     {
36         cin >> arr[i];
37         sum += arr[i];
38     }
39     heapSort(arr, N);
40     cout << "Sorted Response Times:\n";
41     for(int i = 0; i < N; i++)
42         cout << arr[i] << " ";
43
44     cout << endl;
45
46     cout << "Fastest Response Time: " << arr[0] << endl;
47     cout << "Slowest Response Time: " << arr[N-1] << endl;
48     float average = (float)sum / N;
49     cout << "Average Response Time: " << average << endl;
50     int count = 0;
```

5 65 76 456 465 6865
Fastest Response Time: 5
Slowest Response Time: 6865
Average Response Time: 1322
Cases Faster Than Average: 5
Percentage: 83.3333%

PS (thisisdurgeshkumar.com): C:\Users\pc\OneDrive\Desktop\Module\output> cd 'c:\Users\pc\OneDrive\Desktop\Module\output'

```
1 QuickSort.cpp > quickSort(int [], int, int)
1  #include <iostream>
2  using namespace std;
3
4  int partition(int arr[], int low, int high)
5  {
6      int pivot = arr[high];
7      int i = low - 1;
8      for(int j = low; j < high; j++)
9      {
10         if(arr[j] < pivot)
11         {
12             i++;
13             swap(arr[i], arr[j]);
14         }
15     }
16     swap(arr[i + 1], arr[high]);
17     return i + 1;
18 }
19
20 void quickSort(int arr[], int low, int high)
21 {
22     if(low < high)
23     {
24         ...
```

34
765
56
678
76
34 56 76 567 656 678 765

PS (thisisdurgeshkumar.com): C:\Users\pc\OneDrive\Desktop\Module\output>

```
2 QuickSort.cpp > partition(int [], int, int)
1  #include <iostream>
2  using namespace std;
3  int partition(int arr[], int low, int high)
4  {
5      int pivot = arr[high];
6      int i = low - 1;
7      for(int j = low; j < high; j++)
8      {
9          if(arr[j] > pivot) // Descending
10          {
11              i++;
12              swap(arr[i], arr[j]);
13          }
14      }
15      swap(arr[i + 1], arr[high]);
16      return i + 1;
17 }
18 void quickSort(int arr[], int low, int high)
19 {
20     if(low < high)
21     {
22         int pi = partition(arr, low, high);
23
24         quickSort(arr, low, pi - 1);
25         ...
```

Sorted Trade Values:
5454 4556 3545 66 65 45
Top 5 Values:
5454 4556 3545 66 65
Average of Top 5: 2737.2
Count Greater Than Overall Average: 3

PS (thisisdurgeshkumar.com): C:\Users\pc\OneDrive\Desktop\Module\output> cd 'c:\Users\pc\OneDrive\Desktop\Module\output'

① Merge Sort

Space $\rightarrow O(n)$
 Time $\rightarrow O(n \log n)$
 Best $\rightarrow O(n \log n)$
 Avg $\rightarrow O(n \log n)$
 Worst $\rightarrow O(n \log n)$

Algorithm:

- 1. Read n
- 2. Read the array.
- 3. Divide the array in 2 parts.
- 4. Recursively sort both halves.
- 5. Merge two sorted halves.
- 6. Print the sorted array.

Median $\rightarrow \text{arr}[N/2]$

Difference $\rightarrow \text{max} - \text{min}$
 $\text{arr}[N-1] - \text{arr}[0]$

② Heap Sort

Time $\rightarrow O(N \log N)$
 Space $\rightarrow O(1)$

- 1. Read n
- 2. Read the array.
- 3. Build a max heap.
- 4. Swap the root with the last element.
- 5. Reduce heap size by 1.
- 6. Heapify the root.
- 7. Repeat until the heap size becomes 1.
- 8. Print the sorted array.

Left Child $\rightarrow 2*i + 1$

Right Child $\rightarrow 2*i + 2$

Parent $\rightarrow (i-1)/2$

③ Quick Sort - $T_c \rightarrow O(N \log N) \rightarrow$ Best & Avg.
 $SC \rightarrow O(\log N) \rightarrow$ Worst.

- Read N

- Read the array

↓ select the last element as pivot

! place all larger element to the left of the pivot.

! place all larger elements to the right